

Linear__Regression__USN

September 7, 2026

0.1 Implementation of Linear regression using python

Documentation https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

```
[ ]: #import required libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

[ ]: df=pd.read_csv("weight-height.csv")
df

[ ]: df.describe()

[ ]: from sklearn.preprocessing import MinMaxScaler
numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns
scaler = MinMaxScaler()
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

[ ]: df.describe()
```

0.2 Prepare dependant and independant variables

```
[ ]: X =df[['Height']].to_numpy()
Y =df[['Weight']].to_numpy()

[ ]: Y.shape
```

0.3 split the data into training and test set

```
[ ]: X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2,
↳random_state=42)

[ ]: # Create a linear regression model

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
model = LinearRegression()
```

Note: Learning more about Linear Regression model here: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

```
[ ]: # Fit the model to the data
import time
start = time.time()
model.fit(X_train, y_train)
print('Time taken', time.time()-start)
```

```
[ ]: y_pred = model.predict(X_test)
```

```
[ ]: mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("Mean Squared Error (MSE):", mse)
print("R-squared (R2):", r2)

# Print model weights
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_[0])
```

```
[ ]: import matplotlib.pyplot as plt
plt.scatter(X_test[:10], y_test[:10], color='blue', label='Data points')
plt.plot(X_test[:10], y_pred[:10], color='red', label='Regression line')
plt.xlabel('Height')
plt.ylabel('Weight')
plt.legend()
plt.show()
```

```
[ ]: df1 = pd.read_csv('weight-height.csv')
```

```
[ ]: # df1['Height'].values.shape
X = df1['Height'].values.reshape(-1, 1)
X.shape
```

```
[ ]: y = df1['Weight'].values
```

```
[ ]: X = (X - np.mean(X, axis=0)) / np.std(X, axis=0)
y = (y - np.mean(y, axis=0)) / np.std(y, axis=0)
```

```
[ ]: #add another feature to X corresponding to the bias
X = np.hstack((np.ones((X.shape[0], 1)), X))
```

```
[ ]: X.shape
```

```
[ ]: #weight or parameter initialization
theta = np.zeros(X.shape[1])

# Hyperparameters
```

```

learning_rate = 0.01
num_iterations = 1000

# Gradient Descent
mse_history = [] # To store the mean squared error at each iteration

```

```

[ ]: y_pred = np.dot(X, theta)
s=y_pred-y
s.shape

```

```

[ ]: for i in range(num_iterations):
    # Predictions
    y_pred = np.dot(X, theta)

    # Calculate the mean squared error
    mse = mean_squared_error(y, y_pred)
    mse_history.append(mse)

    # Print the MSE at each iteration
    print(f"Iteration {i+1}: MSE = {mse}")

    # Gradient calculation
    gradient = np.dot(X.T, (y_pred - y)) / len(y)

    # Update parameters
    theta -= learning_rate * gradient

```

```

[ ]: # Plot the MSE over iterations
plt.plot(range(num_iterations), mse_history)
plt.xlabel('Iterations')
plt.ylabel('Mean Squared Error (MSE)')
plt.title('MSE vs Iterations')
plt.show()

```

Task 1 You are required to fit a linear function on the Food_delivery_times dataset

Description about the dataset: This dataset is designed for predicting food delivery times based on various influencing factors such as distance, weather, traffic conditions, and Preparation time.

1. Feature selection: Consider Food_delivery_time is affected by four features (attributes) given in the dataset, which are Distance_km, Preparation_Time_min, Traffic, and Weather 2. Since Traffic and Weather have categorical values, you need to convert them into numerical values using categorical encoding methods. 3. Now, fit a linear regression model considering Distance_km, Preparation_Time_min, Traffic, and Weather as independent variables and Delivery_Time_min as dependent variable.

Task 2 You are given the weight-height.csv dataset containing:

Height

Weight

Gender

Your task is to build a Logistic Regression classifier to predict the Gender of a person based on their Height and Weight.

[]: